LINUX PROCESS_LAB

Aditi Biju

261100690027

Exercise 1: Display all running processes with their PID, PPID, owner, CPU usage, memory usage, and command.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -eo pid,ppid,user,%cpu,%mem,cmd
    PID    PPID USER     %CPU %MEM CMD
      1       0 root      0.0  0.5 /sbin/init splash
      2       0 root      0.0  0.0 [kthreadd]
      3       2 root      0.0  0.0 [pool_workqueue_release]
```

List only the processes belonging to the current user.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -u $USER -o pid,ppid,user,%cpu,%mem,cmd
    PID    PPID USER     %CPU %MEM CMD
   1324       1 kali      0.0  0.4 /usr/lib/systemd/systemd --user
   1326    1324 kali      0.0  0.1 (sd-pam)
```

Identify the five processes consuming the most CPU.

Identify the five processes consuming the most memory.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head -6
    PID    PPID USER     %CPU %MEM CMD
   1481    1365 kali      0.5  4.8 xfwm4
   1109    1064 root      0.3  5.1 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt
7 -novtswitch
   1587    1526 kali      0.2  1.6 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4
/panel/plugins/libcpugraph.so 13 16777228 cpugraph CPU Graph Graphical representation of the CPU load
   1905       1 kali      0.2  1.5 /usr/bin/vmtoolsd -n vmusr --blockFd 3
    605       1 root      0.2  0.3 /usr/bin/vmtoolsd

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem | head -6
    PID    PPID USER     %CPU %MEM CMD
   1109    1064 root      0.3  5.1 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt
7 -novtswitch
   1481    1365 kali      0.5  4.8 xfwm4
   2228       1 kali      0.0  2.3 /usr/bin/qterminal
   1869    1365 kali      0.0  2.1 /usr/bin/python3 /usr/bin/blueman-applet
   1543    1365 kali      0.0  2.0 xfdesktop
```

Explain why high CPU usage alone does not confirm malicious activity.

High CPU usage does not always indicate malicious activity. Legitimate tasks such as compiling programs, video encoding, antivirus scans and software updates can consume high CPU resources.

To identify a suspicious process, we should also check its owner, parent process, binary location, start time, network connections and whether its name or path looks suspicious.

Exercise 2: Parent and child processes Write a shell script that starts two background processes and displays their PIDs.

```
#!/bin/bash

echo "Script PID: $$"
echo "Script PPID: $PPID"

sleep 300 &
echo "Child 1 PID: $!"

sleep 400 &
echo "Child 2 PID: $!"

echo "Children of this script:"
ps -o pid,ppid,cmd --ppid $$

wait
```

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ Script PID: 96749
Script PPID: 2265
Child 1 PID: 96751
Child 2 PID: 96752
Children of this script:
    PID    PPID CMD
  96751   96749 sleep 300
  96752   96749 sleep 400
  96753   96749 ps -o pid,ppid,cmd --ppid 96749
```

Identify the PID and PPID of the shell script.

Display the child processes created by the script.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -p 96749  -o pid,ppid,cmd
    PID    PPID CMD
  96749    2265 /bin/bash ./parent_child.sh

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -o pid,ppid,cmd --ppid 96749
    PID    PPID CMD
  96751   96749 sleep 300
  96752   96749 sleep 400
```

Explain the purpose of $$, $!, &, and wait.

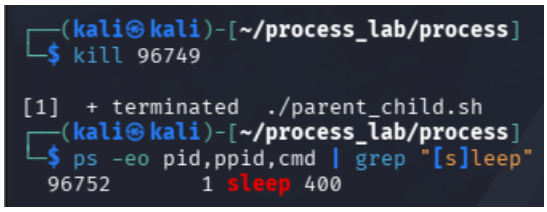
$$ - Displays the PID of the current shell or script.

$! - Displays the PID of the most recently started background process.

& - Runs a command in the background.

Wait - Waits for background child processes to finish.

Observe what happens to the child processes when the parent script terminates.



Exercise 3: Process control using kill

Start a background process and record its PID.

Suspend the process and verify its state.

Resume the process and verify its state.

Terminate the process gracefully.

Verify that the process has terminated.



Explain the differences between SIGSTOP, SIGCONT, SIGTERM, and SIGKILL.

Signal	Explanation
SIGSTOP	Pauses a process. It cannot be caught or ignored.
SIGCONT	Resumes a stopped process.
SIGTERM	Requests a process to terminate gracefully.
SIGKILL	Immediately terminates a process. It cannot be caught or ignored.

Exercise 4: Graceful and forced termination

Write a monitoring script that periodically records a timestamp in a log file.

Configure the script to record a shutdown message when it receives SIGTERM.

```
#!/bin/bash

LOG="$HOME/process_lab/monitor.log"

trap 'echo "$(date) - SIGTERM received, shutting down gracefully" >> "$LOG"; exit 0' TERM

echo "$(date) - Started, PID $$" >> "$LOG"

while true
do
    echo "$(date) - heartbeat" >> "$LOG"
    sleep 5 &
    wait $!
done
```

Terminate the script using SIGTERM and inspect the log.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ chmod +x monitor.sh

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ./monitor.sh &
[1] 102172

┌──(kali㉿kali)-[~/process_lab/process]
└─$ MPID=$!

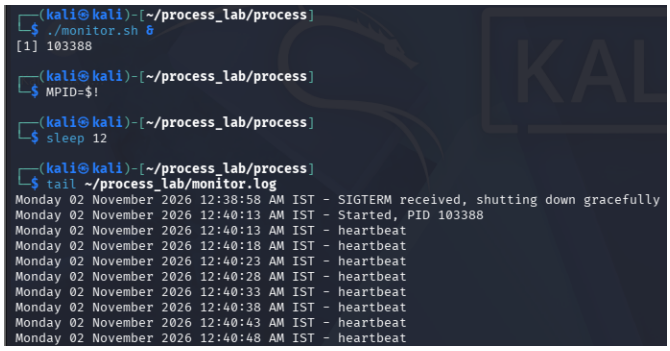
┌──(kali㉿kali)-[~/process_lab/process]
└─$ sleep 12

┌──(kali㉿kali)-[~/process_lab/process]
└─$ kill -TERM $MPID

[1]  + done       ./monitor.sh
┌──(kali㉿kali)-[~/process_lab/process]
└─$ cat monitor.log
cat: monitor.log: No such file or directory

┌──(kali㉿kali)-[~/process_lab/process]
└─$ cat ~/process_lab/monitor.log
Monday 02 November 2026 12:37:55 AM IST - Started, PID 102172
Monday 02 November 2026 12:37:55 AM IST - heartbeat
Monday 02 November 2026 12:38:00 AM IST - heartbeat
```

Restart the script and terminate it using SIGKILL.



Compare the results and explain why they differ.

SIGTERM can be caught by a process, so the trap handler executes and records the shutdown message in the log file.

SIGKILL cannot be caught or handled. The kernel immediately terminates the process, so the cleanup code does not execute.

Exercise 5: Multiple processes using killall

Start three instances of a lab program with a unique process name.



Display the PID and owner of each instance.

ps -C labproc -o pid,user,cmd

Terminate one instance using its PID.

kill <ONE_PID>

Terminate the remaining instances using killall.

kill <ONE_PID>

Verify that all instances have terminated.

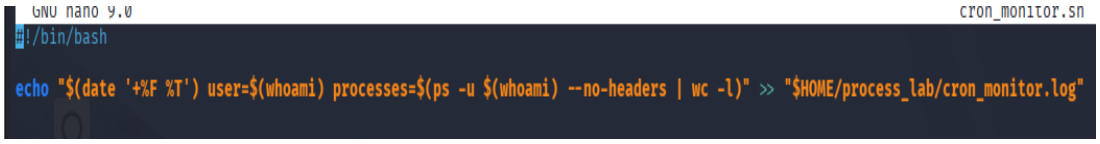
pgrep -a labproc

Explain the risk of using killall with a common process name.

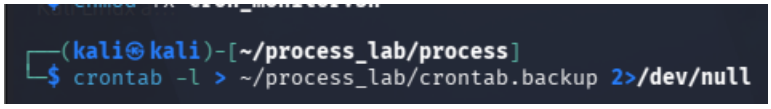
The killall command terminates all processes matching the specified name. If we use a common process name such as Python, Bash, Java or SSHD, it may terminate unrelated or important processes. Therefore, we should use unique process names and verify the processes before terminating them.

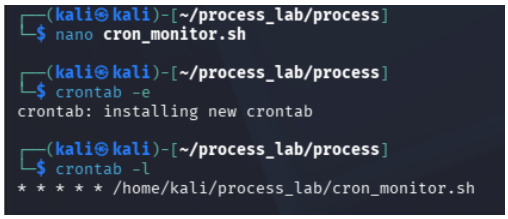
Exercise 6: Recurring process monitoring using cron

Write a shell script that records the current date, time, username, and number of processes owned by the current user.



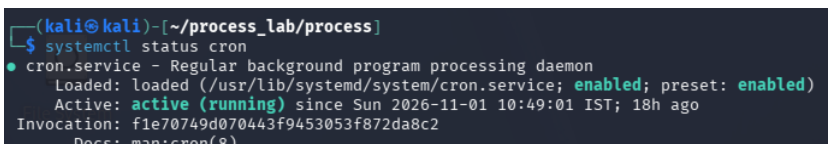
Schedule the script to run every minute.





Verify that the scheduled task executes successfully.

systemctl status cron tail -f ~/process_lab/cron_monitor.log



```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ cat ~/process_lab/cron_monitor.log
2026-11-02 05:23:30 user=kali processes=69

┌──(kali㉿kali)-[~/process_lab/process]
└─$ tail ~/process_lab/cron_monitor.sh
#!/bin/bash

echo "$(date '+%F %T') user=$(whoami) processes=$(ps -u $(whoami) --no-headers | wc -l)" >> "$HOME/process_lab/cron_
monitor.log"
```

Start additional background processes and observe changes in the log.

Sleep 500&

Modify the schedule to run every five minutes.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -l
* * * * * /home/kali/process_lab/cron_monitor.sh
```

Remove the exercise entry without deleting other cron entries.

crontab -l | grep -v "cron_monitor.sh" | crontab - crontab -l

Exercise 7: Cron scheduling

Write cron entries for the following requirements:

Check failed-login logs every five minutes.

Capture a process snapshot at the beginning of every hour.

Back up security logs daily at 2:30 AM.

Generate a security report every weekday at 9:00 AM.

Perform a file-integrity check every Sunday at 3:00 AM.

Run a monitoring script every 15 minutes.

Generate a weekly report every Monday at 8:30 AM.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -e
crontab: installing new crontab

┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -e
crontab: installing new crontab

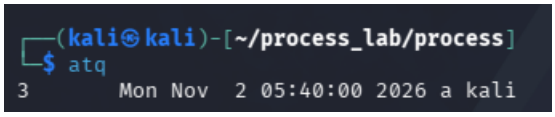
┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -l
*/5 * * * * /usr/local/bin/check_failed_logins.sh
0 * * * * /usr/local/bin/process_snapshot.sh
30 2 * * * /usr/local/bin/backup_security_logs.sh
0 9 * * 1-5 /usr/local/bin/security_report.sh
0 3 * * 0 /usr/local/bin/integrity_check.sh
*/15 * * * * /usr/local/bin/monitor.sh
30 8 * * 1 /usr/local/bin/weekly_report.sh
```

Exercise 8: One-time scheduling using at

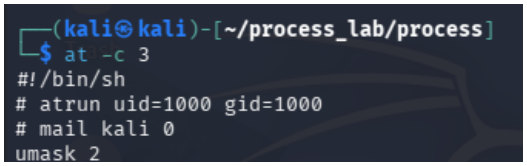
Schedule a task to record the current date and process list two minutes from now.

echo "date >> $HOME/process_lab/at_output.txt; ps aux >>
$HOME/process_lab/at_output.txt" | at now + 2 minutes

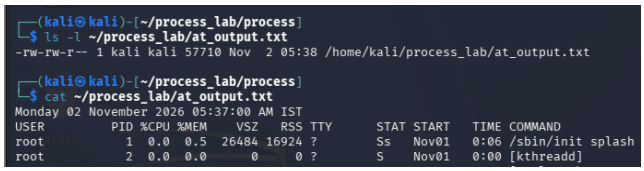
Display all pending jobs.



Inspect the commands associated with the scheduled job.



Verify that the output file is created after execution.



Explain why the completed job no longer appears in the pending-job list.

The at command is used for one-time scheduling. After the scheduled task executes, it is automatically removed from the pending-job queue. Therefore, completed jobs no longer appear when we use the atq command.

Exercise 9: Cancelling scheduled jobs

Schedule a harmless task to execute ten minutes from now.

Identify its job number.

Cancel the scheduled task.

Verify that the task has been removed.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ echo "echo hello >> $HOME/process_lab/harmless.txt" | at now + 10 minutes
warning: commands will be executed using /bin/sh
job 4 at Mon Nov  2 06:00:00 2026

┌──(kali㉿kali)-[~/process_lab/process]
└─$ atq
4       Mon Nov  2 06:00:00 2026 a kali

┌──(kali㉿kali)-[~/process_lab/process]
└─$ atrm 4

┌──(kali㉿kali)-[~/process_lab/process]
└─$ atq
```

Explain the difference between cancelling an at job and removing a cron entry.

An at job is a one-time scheduled task identified by a job number. It can be cancelled using the atrm command.

A cron entry is a recurring scheduled task stored in the user's crontab. It must be removed by editing the crontab and deleting the corresponding entry. Otherwise, it continues executing according to its schedule.

Exercise 10: Incident-response scenario

An unfamiliar lab process has multiple running instances and a scheduled task that will restart it.

Identify all instances of the lab process and record their PIDs and owners.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ chmod +x labproc

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ./labproc 900 &
[2] 223765

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ./labproc 900 &
[3] 223829

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ./labproc 900 &
[4] 223911

┌──(kali㉿kali)-[~/process_lab/process]
└─$ echo "$HOME/process_lab/labproc 900 &" | at now + 10 minutes
warning: commands will be executed using /bin/sh
job 5 at Mon Nov  2 06:04:00 2026

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -eo pid,ppid,user,stat,cmd | grep '[l]abproc'
 223765    2265 kali     SN   /bin/bash ./labproc 900
 223829    2265 kali     SN   /bin/bash ./labproc 900
 223911    2265 kali     SN   /bin/bash ./labproc 900
```

Suspend one instance and verify its state.

Resume the suspended instance.

Terminate all instances of the lab process.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ kill -SIGSTOP 223765
[2]  + suspended (signal)  ./labproc 900

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -o pid,stat,cmd -p 223765
    PID STAT CMD
 223765 TN   /bin/bash ./labproc 900

┌──(kali㉿kali)-[~/process_lab/process]
└─$ kill -SIGCONT 223765

┌──(kali㉿kali)-[~/process_lab/process]
└─$ ps -o pid,stat,cmd -p 223765
    PID STAT CMD
 223765 SN   /bin/bash ./labproc 900

┌──(kali㉿kali)-[~/process_lab/process]
└─$ pkill -TERM -x labproc

[3]    done       ./labproc 900
[2]  + done       ./labproc 900
[4]  + done       ./labproc 900
```

Identify and inspect the scheduled restart task.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ atq
5       Mon Nov  2 06:04:00 2026 a kali

┌──(kali㉿kali)-[~/process_lab/process]
└─$ at -c 5
#!/bin/sh
# atrun uid=1000 gid=1000
# mail kali 0
```

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -l
*/5 * * * * /usr/local/bin/check_failed_logins.sh
0 * * * * /usr/local/bin/process_snapshot.sh
30 2 * * * /usr/local/bin/backup_security_logs.sh
0 9 * * 1-5 /usr/local/bin/security_report.sh
0 3 * * 0 /usr/local/bin/integrity_check.sh
*/15 * * * * /usr/local/bin/monitor.sh
30 8 * * 1 /usr/local/bin/weekly_report.sh

┌──(kali㉿kali)-[~/process_lab/process]
└─$ sudo ls /etc/cron.d /var/spool/cron
[sudo] password for kali:
/etc/cron.d:
e2scrub_all  john  php  sysstat

/var/spool/cron:
atjobs  atspool  crontabs
```

Cancel the scheduled restart.

Verify that no instances or restart jobs remain.

```
┌──(kali㉿kali)-[~/process_lab/process]
└─$ atrm 5

┌──(kali㉿kali)-[~/process_lab/process]
└─$ pgrep -a -x labproc

┌──(kali㉿kali)-[~/process_lab/process]
└─$ atq

┌──(kali㉿kali)-[~/process_lab/process]
└─$ crontab -l
*/5 * * * * /usr/local/bin/check_failed_logins.sh
0 * * * * /usr/local/bin/process_snapshot.sh
30 2 * * * /usr/local/bin/backup_security_logs.sh
0 9 * * 1-5 /usr/local/bin/security_report.sh
0 3 * * 0 /usr/local/bin/integrity_check.sh
*/15 * * * * /usr/local/bin/monitor.sh
30 8 * * 1 /usr/local/bin/weekly_report.sh
```